

\$group (aggregation stage)

Definition

`$group` The `$group` stage combines multiple documents with the same field, fields, or expression into a single document according to a group key. The result is one document per unique group key.

A group key is often a field, or group of fields. The group key can also be the result of an expression. Use the `_id` field in the `$group` pipeline stage to set the group key. See below for usage examples.

In the `$group` stage output, the `_id` field is set to the group key for that document.

The output documents can also contain additional fields that are set using accumulator expressions.

`$group` does *not* order its output documents.

Compatibility

`$group` You can use `$group` for deployments hosted in the following environments:

- MongoDB Atlas: The fully managed service for MongoDB deployments in the cloud
- MongoDB Enterprise: The subscription-based, self-managed version of MongoDB
- MongoDB Community: The source-available, free-to-use, and self-managed version of MongoDB

Syntax

The `$group` stage has the following prototype form:

```
{
  $group:
  {
    _id: <expression>, // Group key
    <field1>: { <accumulator1> : <expression1> },
    ...
  }
}
```

Field	Description
`_id`	*Required.* The `_id` expression specifies the group key. If you specify an `_id` value of null, or any other constant value, the `\$group` stage returns a single document that aggregates values across all of the input documents. See the Group by Null example.
`field`	*Optional.* Computed using the accumulator operators.

The `_id` and the accumulator operators can accept any valid `expression` . For more information on expressions, see Expressions.

Considerations

Performance

`$group` `$group` is a blocking stage, which causes the pipeline to wait for all input data to be retrieved for the blocking stage before processing the data. A blocking stage may reduce performance because it reduces parallel processing for a pipeline with multiple stages. A blocking stage may also use substantial amounts of memory for large data sets.

Accumulator Operator

The `<accumulator>` operator must be one of the following accumulator operators:

Name	Description
`\$accumulator`	Returns the result of a user-defined accumulator function.
`\$addToSet`	Returns an array of <i>*unique*</i> expression values for each group. Order of the array elements is undefined. Available in the `\$setWindowFields` stage.
`\$avg`	Returns an average of numerical values. Ignores non-numeric values. Available in the `\$setWindowFields` stage.
`\$bottom`	Returns the bottom element within a group according to the specified sort order. Available in the `\$group` and `\$setWindowFields` stages.
`\$bottomN`	Returns an aggregation of the bottom `n` fields within a group, according to the specified sort order. Available in the `\$group` and `\$setWindowFields` stages.
`\$concatArrays`	Returns a single array that combines the elements of two or more arrays.
`\$count`	Returns the number of documents in a group. Distinct from the `\$count` pipeline stage. Available in the `\$group` and `\$setWindowFields` stages.
`\$first`	Returns the result of an expression for the first document in a group. Available in the `\$setWindowFields` stage.
`\$firstN`	Returns an aggregation of the first `n` elements within a group. Only meaningful when documents are in a defined order. Distinct from the `\$firstN` array operator. Available in the `\$group`, expression and `\$setWindowFields` stages.
`\$last`	Returns the result of an expression for the last document in a group. Available in the `\$setWindowFields` stage.
`\$lastN`	Returns an aggregation of the last `n` elements within a group. Only meaningful when documents are in a defined order. Distinct from the `\$lastN` array operator. Available in the `\$group`, expression and `\$setWindowFields` stages.
`\$max`	Returns the highest expression value for each group. Available in the `\$setWindowFields` stage.
`\$maxN`	Returns an aggregation of the `n` maximum valued elements in a group. Distinct from the `\$maxN` array operator. Available in `\$group`,

	`\$setWindowFields` and as an expression.
`\$median`	Returns an approximation of the median, the 50th percentile, as a scalar value. This operator is available as an accumulator in these stages: - `\$group` - `\$setWindowFields` It is also available as an aggregation expression.
`\$mergeObjects`	Returns a document created by combining the input documents for each group.
`\$min`	Returns the lowest expression value for each group. Available in the `\$setWindowFields` stage.
`\$minN`	Returns an aggregation of the `n` minimum valued elements in a group. Distinct from the `\$minN` array operator. Available in `\$group`, `\$setWindowFields` and as an expression.
`\$percentile`	Returns an array of scalar values that correspond to specified percentile values. This operator is available as an accumulator in these stages: - `\$group` - `\$setWindowFields` It is also available as an aggregation expression.
`\$push`	Returns an array of expression values for documents in each group. Available in the `\$setWindowFields` stage.
`\$setUnion`	Takes two or more arrays and returns an array containing the elements that appear in any input array.
`\$stdDevPop`	Returns the population standard deviation of the input values. Available in the `\$setWindowFields` stage.
`\$stdDevSamp`	Returns the sample standard deviation of the input values. Available in the `\$setWindowFields` stage.
`\$sum`	Returns a sum of numerical values. Ignores non-numeric values. Available in the `\$setWindowFields` stage.
`\$top`	Returns the top element within a group according to the specified sort order. Available in the `\$group` and `\$setWindowFields` stages.
`\$topN`	Returns an aggregation of the top `n` fields within a group, according to the specified sort order. Available in the `\$group` and `\$setWindowFields` stages.

\$group and Memory Restrictions

If the `$group` stage exceeds 100 megabytes of RAM, MongoDB writes data to temporary files. However, if the `allowDiskUse` option is set to `false`, `$group` returns an error. For more information, refer to [Aggregation Pipeline Limits](#).

\$group Performance Optimizations

This section describes optimizations to improve the performance of `$group`. There are optimizations that you can make manually and optimizations MongoDB makes internally.

Optimization to Return the First or Last Document of Each Group

If a pipeline `sorts` and `groups` by the same field and the `$group` stage only uses the `$first` or `$last` accumulator operator, consider adding an index on the grouped field which matches the sort order. In some cases, the `$group` stage can use the index to quickly find the first or last document of each group.

If a collection named `foo` contains an index `{ x: 1, y: 1 }`, the following pipeline can use that index to find the first document of each group:

```
db.foo.aggregate([
  {
    $sort:{ x : 1, y : 1 }
  },
  {
    $group: {
      _id: { x : "$x" },
      y: { $first : "$y" }
    }
  }
])
```

Slot-Based Query Execution Engine

Starting in version 5.2, MongoDB uses the slot-based execution query engine to execute `$group` stages if either:

- `$group` is the first stage in the pipeline.
- All preceding stages in the pipeline can also be executed by the slot-based execution engine.

For more information, see `$group` Optimization.

Examples

Group and Count Documents By Field

In `mongosh`, create a sample collection named `sales` with the following documents:

```
db.sales.insertMany([
  { "_id" : 1, "item" : "abc", "price" : Decimal128("10"), "quantity" : Int32("2") },
  { "_id" : 2, "item" : "jkl", "price" : Decimal128("20"), "quantity" : Int32("1") },
  { "_id" : 3, "item" : "xyz", "price" : Decimal128("5"), "quantity" : Int32("3") },
  { "_id" : 4, "item" : "xyz", "price" : Decimal128("5"), "quantity" : Int32("2") },
  { "_id" : 5, "item" : "abc", "price" : Decimal128("10"), "quantity" : Int32("2") },
  { "_id" : 6, "item" : "def", "price" : Decimal128("7.5"), "quantity" : Int32("1") },
  { "_id" : 7, "item" : "def", "price" : Decimal128("7.5"), "quantity" : Int32("1") },
  { "_id" : 8, "item" : "abc", "price" : Decimal128("10"), "quantity" : Int32("1") }
])
```

The following aggregation operation uses the `$group` stage to group documents in the `sales` collection by the `price` field. One group contains the items with a price greater than or equal to 10. The second group contains the number of items with a price less than 10. The pipeline then counts the number of documents in each group.

```
db.sales.aggregate([
  {
    $group: {
      _id: {
        $cond: {
          if: { $gte: [ "$price", Decimal128("10") ] },
          then: "Price >= 10",
          else: "Price < 10"
        }
      },
    },
    count: { $sum: 1 }
  }
])
```

The operation returns the following result:

```
{ _id: 'Price >= 10', count: 4 }, { _id: 'Price < 10', count: 4 }
```

This aggregation operation is equivalent to the following SQL statement:

```
SELECT
  CASE
    WHEN price >= 10 THEN 'Price >= 10'
    ELSE 'Price < 10'
  END AS price_group,
  COUNT(*) AS count
FROM
  sales
GROUP BY
  price_group;
```

- `$count`
- `$count` (aggregation accumulator)

Retrieve Distinct Values

The following aggregation operation uses the `$group` stage to retrieve the distinct item values from the `sales` collection:

```
db.sales.aggregate( [ { $group : { _id : "$item" } } ] )
```

The operation returns the following result:

```
{ "_id" : "abc" }
{ "_id" : "jkl" }
{ "_id" : "def" }
{ "_id" : "xyz" }
```

For example, `$group` operations of the following form can result in a `DISTINCT_SCAN` :

```
{ $group : { _id : "$<field>" } }
```

For more information on behavior for retrieving distinct values, see the `distinct` command behavior.

To see whether your operation results in a `DISTINCT_SCAN`, check your operation's explain results.

Group by Item Having

The following aggregation operation groups documents by the `item` field, calculating the total sale amount per item and returning only the items with total sale amount greater than or equal to 100:

```
db.sales.aggregate([
  // First Stage
  {
    $group :
    {
      _id : "$item",
      totalSaleAmount: { $sum: { $multiply: [ "$price", "$quantity" ] } }
    }
  },
  // Second Stage
  {
    $match: { "totalSaleAmount": { $gte: 100 } }
  }
])
```

The `$group` stage groups the documents by `item` to retrieve the distinct item values. This stage returns the `totalSaleAmount` for each item.

The `$match` stage filters the resulting documents to only return items with a `totalSaleAmount` greater than or equal to 100.

The operation returns the following result:

```
{ "_id" : "abc", "totalSaleAmount" : Decimal128("170") }
{ "_id" : "xyz", "totalSaleAmount" : Decimal128("150") }
{ "_id" : "def", "totalSaleAmount" : Decimal128("112.5") }
```

This aggregation operation is equivalent to the following SQL statement:


```
SELECT item,
       Sum(( price * quantity )) AS totalSaleAmount
FROM   sales
GROUP  BY item
HAVING totalSaleAmount >= 100
```

`$match`

Calculate Count, Sum, and Average

In `mongosh`, create a sample collection named `sales` with the following documents:

```
db.sales.insertMany([
  { "_id" : 1, "item" : "abc", "price" : Decimal128("10"), "quantity" : Int32("2") },
  { "_id" : 2, "item" : "jkl", "price" : Decimal128("20"), "quantity" : Int32("1") },
  { "_id" : 3, "item" : "xyz", "price" : Decimal128("5"), "quantity" : Int32("3") },
  { "_id" : 4, "item" : "xyz", "price" : Decimal128("5"), "quantity" : Int32("2") },
  { "_id" : 5, "item" : "abc", "price" : Decimal128("10"), "quantity" : Int32("2") },
  { "_id" : 6, "item" : "def", "price" : Decimal128("7.5"), "quantity" : Int32("1") },
  { "_id" : 7, "item" : "def", "price" : Decimal128("7.5"), "quantity" : Int32("1") },
  { "_id" : 8, "item" : "abc", "price" : Decimal128("10"), "quantity" : Int32("2") }
])
```

Group by Day of the Year

The following pipeline calculates the total sales amount, average sales quantity, and sale count for each day in the year 2014:

```
db.sales.aggregate([
  // First Stage
  {
    $match : { "date": { $gte: new ISODate("2014-01-01"), $lt: new ISODate("2015-01-01") } },
  },
  // Second Stage
  {
    $group : {
      _id : { $dateToString: { format: "%Y-%m-%d", date: "$date" } },
      totalSaleAmount: { $sum: { $multiply: [ "$price", "$quantity" ] } },
      averageQuantity: { $avg: "$quantity" },
      count: { $sum: 1 }
    }
  },
  // Third Stage
  {
    $sort : { totalSaleAmount: -1 }
  }
])
```

The `$match` stage filters the documents to only pass documents from the year 2014 to the next stage.

The `$group` stage groups the documents by date and calculates the total sale amount, average quantity, and total count of the documents in each group.

The `$sort` stage sorts the results by the total sale amount for each group in descending order.

The operation returns the following results:

```
{
  "_id" : "2014-04-04",
  "totalSaleAmount" : Decimal128("200"),
  "averageQuantity" : 15, "count" : 2
}
{
  "_id" : "2014-03-15",
  "totalSaleAmount" : Decimal128("50"),
  "averageQuantity" : 10, "count" : 1
}
{
  "_id" : "2014-03-01",
  "totalSaleAmount" : Decimal128("40"),
  "averageQuantity" : 1.5, "count" : 2
}
```

This aggregation operation is equivalent to the following SQL statement:

```
SELECT date,
       Sum(( price * quantity )) AS totalSaleAmount,
       Avg(quantity)             AS averageQuantity,
       Count(*)                  AS Count
FROM   sales
WHERE  date >= '01/01/2014' AND date < '01/01/2015'
GROUP BY date
ORDER BY totalSaleAmount DESC
```

- `$match`
- `$sort`
- `db.collection.countDocuments()` which wraps the `$group` aggregation stage with a `$sum` expression.

Group by `null`

The following aggregation operation specifies a group `_id` of `null`, calculating the total sale amount, average quantity, and count of *all* documents in the collection.

```

db.sales.aggregate([
  {
    $group : {
      _id : null,
      totalSaleAmount: { $sum: { $multiply: [ "$price", "$quantity" ] } },
      averageQuantity: { $avg: "$quantity" },
      count: { $sum: 1 }
    }
  }
])

```

The operation returns the following result:

```

{
  "_id" : null,
  "totalSaleAmount" : Decimal128("452.5"),
  "averageQuantity" : 7.875,
  "count" : 8
}

```

This aggregation operation is equivalent to the following SQL statement:

```

SELECT Sum(price * quantity) AS totalSaleAmount,
       Avg(quantity)         AS averageQuantity,
       Count(*)              AS Count
FROM   sales

```

- `$count`
- `db.collection.countDocuments()` which wraps the `$group` aggregation stage with a `$sum` expression.

Pivot Data

In `mongosh`, create a sample collection named `books` with the following documents:

```
db.books.insertMany([
  { "_id" : 8751, "title" : "The Banquet", "author" : "Dante", "copies" : 2 },
  { "_id" : 8752, "title" : "Divine Comedy", "author" : "Dante", "copies" : 1 },
  { "_id" : 8645, "title" : "Eclogues", "author" : "Dante", "copies" : 2 },
  { "_id" : 7000, "title" : "The Odyssey", "author" : "Homer", "copies" : 10 },
  { "_id" : 7020, "title" : "Iliad", "author" : "Homer", "copies" : 10 }
])
```

Group title by author

The following aggregation operation pivots the data in the `books` collection to have titles grouped by authors.

```
db.books.aggregate([
  { $group : { _id : "$author", books: { $push: "$title" } } }
])
```

The operation returns the following documents:

```
{ "_id" : "Homer", "books" : [ "The Odyssey", "Iliad" ] }
{ "_id" : "Dante", "books" : [ "The Banquet", "Divine Comedy", "Eclogues" ] }
```

Group Documents by author

The following aggregation operation groups documents by `author` :

```

db.books.aggregate([
  // First Stage
  {
    $group : { _id : "$author", books: { $push: "$$ROOT" } }
  },
  // Second Stage
  {
    $addFields:
    {
      totalCopies : { $sum: "$books.copies" }
    }
  }
])

```

`$group` uses the `$$ROOT` system variable to group the entire documents by authors. This stage passes the following documents to the next stage:

```

{ "_id" : "Homer",
  "books" :
  [
    { "_id" : 7000, "title" : "The Odyssey", "author" : "Homer", "copies" : 10 },
    { "_id" : 7020, "title" : "Iliad", "author" : "Homer", "copies" : 10 }
  ]
},
{ "_id" : "Dante",
  "books" :
  [
    { "_id" : 8751, "title" : "The Banquet", "author" : "Dante", "copies" : 2 },
    { "_id" : 8752, "title" : "Divine Comedy", "author" : "Dante", "copies" : 2 },
    { "_id" : 8645, "title" : "Eclogues", "author" : "Dante", "copies" : 2 }
  ]
}

```

`$addFields` adds a field to the output containing the total copies of books for each author.

The resulting documents must not exceed the BSON Document Size limit of 16 mebibytes.

The operation returns the following documents:

```

{
  "_id" : "Homer",
  "books" :
    [
      { "_id" : 7000, "title" : "The Odyssey", "author" : "Homer", "copies" : 10 },
      { "_id" : 7020, "title" : "Iliad", "author" : "Homer", "copies" : 10 }
    ],
  "totalCopies" : 20
}

{
  "_id" : "Dante",
  "books" :
    [
      { "_id" : 8751, "title" : "The Banquet", "author" : "Dante", "copies" : 2 },
      { "_id" : 8752, "title" : "Divine Comedy", "author" : "Dante", "copies" : 2 },
      { "_id" : 8645, "title" : "Eclogues", "author" : "Dante", "copies" : 1 }
    ],
  "totalCopies" : 5
}

```

The C# examples on this page use the `sample_mflix` database from the Atlas sample datasets. To learn how to create a free MongoDB Atlas cluster and load the sample datasets, see [Get Started in the MongoDB .NET/C# Driver documentation](#).

The following `Movie` class models the documents in the `sample_mflix.movies` collection:

```

public class Movie
{
    public ObjectId Id { get; set; }

    public int Runtime { get; set; }

    public string Title { get; set; }

    public string Rated { get; set; }

    public List<string> Genres { get; set; }

    public string Plot { get; set; }

    public ImdbData Imdb { get; set; }

    public int Year { get; set; }

    public int Index { get; set; }

    public string[] Comments { get; set; }

    [BsonElement("lastupdated")]
    public DateTime LastUpdated { get; set; }
}

```

The C# classes on this page use Pascal case for their property names, but the field names in the MongoDB collection use camel case. To account for this difference, you can use the following code to register a `ConventionPack` when your application starts:

```

var camelCaseConvention = new ConventionPack { new CamelCaseElementNameConvention() };
ConventionRegistry.Register("CamelCase", camelCaseConvention, type => true);

```

```
$group
```

```
Group()
```

groups documents by the value of their `Rated` field. Each group's rating is shown in a field named `Rating` in each output document. Each output document also contains fields named

`TotalRuntime`, `MedianRuntime`, and `NinetiethPercentileRuntime`, which store the total, median, and 90th percentile runtime values for movies in each group.

To use the MongoDB .NET/C# driver to add a `$group` stage to an aggregation pipeline, call the `Group()` method on a `PipelineDefinition` object.

The following example creates a pipeline stage that groups documents by the value of their `Rated` field. Each group's rating is shown in a field named `Rating` in each output document. Each output document also contains fields named `TotalRuntime`, `MedianRuntime`, and `NinetiethPercentileRuntime`, which store the total, median, and 90th percentile runtime values for movies in each group.

```
var pipeline = new EmptyPipelineDefinition<Movie>()
    .Group(
        id: m => m.Rated,
        group: g => new
        {
            Rating = g.Key,
            TotalRuntime = g.Sum(m => m.Runtime),
            MedianRuntime = g.Select(m => m.Runtime).Median(),
            NinetiethPercentileRuntime = g.Select(m => m.Runtime).Percentile(new
        }
    );
```

The Node.js examples on this page use the `sample_mflix` database from the Atlas sample datasets. To learn how to create a free MongoDB Atlas cluster and load the sample datasets, see [Get Started in the MongoDB Node.js driver documentation](#).

`$group`

groups documents by the value of their `rated` field. Each output document contains a `rating` field that stores each group's rating. Each output document also contains a field named `totalRuntime` that stores the total runtime of all movies in the group

To use the MongoDB Node.js driver to add a `$group` stage to an aggregation pipeline, use the `$group` operator in a pipeline object.

The following example creates a pipeline stage that groups documents by the value of their `rated` field. Each output document contains a `rating` field that stores each group's rating. Each output document also contains a field named `totalRuntime` that stores the total runtime of all movies in the group. The example then runs the aggregation pipeline:

```
const pipeline = [
  {
    $group: {
      _id: "$rated",
      rating: { $first: "$rated" },
      totalRuntime: { $sum: "$runtime" }
    }
  }
];

const cursor = collection.aggregate(pipeline);
return cursor;
```

Learn More

The Group and Total Data tutorial provides an extensive example of the `$group` operator in a common use case.

To learn more about related pipeline stages, see the `$addFields` guide.